

# Informe – Coordinación y Escalabilidad del Sistema

CAROLINA ANABEL RACEDO

Abril, 2026

La arquitectura del sistema está diseñada para procesar flujos de datos masivos garantizando coordinación y escalabilidad. Para soportar el procesamiento concurrente de múltiples clientes sin mezclar estados, cada flujo se identifica unívocamente mediante un **UUID** generado en el **Gateway**, lo que permite que los nodos procesen información de distintos clientes de forma aislada.

En la primera etapa, la escalabilidad se logra distribuyendo los datos a través de una *Work Queue*, permitiendo agregar múltiples réplicas del nodo **Sum** que consumen mensajes de forma balanceada. Sin embargo, esta estrategia introduce un problema de coordinación al finalizar la transmisión: el mensaje de fin de archivo (*EOF*) enviado por el **Gateway** es consumido por una única instancia **Sum**, dejando al resto sin conocimiento del cierre del flujo. Para resolverlo, la instancia que recibe el *EOF* original lo difunde mediante un **Control Exchange**. Cada nodo **Sum** cuenta con un *thread* secundario dedicado exclusivamente a consumir este *exchange*.

Se opta por utilizar *threads* en lugar de procesos, ya que la recepción de mensajes es una tarea *I/O-bound*. Aunque el GIL de Python limita el paralelismo en tareas de CPU, se libera durante operaciones de red, permitiendo concurrencia efectiva. Este enfoque, además, reduce el *overhead* de memoria y evita la complejidad de sincronización mediante *IPC*.

Dado que los mensajes de datos y de control viajan por canales distintos, se pierde el orden de entrega, generando una posible condición de carrera distribuida. Para solucionarlo de forma determinística y sin depender de tiempos arbitrarios, el **Gateway** adjunta la cantidad total de mensajes emitidos en su *EOF*. A su vez, cada nodo **Sum** contabiliza su procesamiento local e informa sus avances al resto mediante el *Control Exchange*. El estado final (y el posterior envío de datos) se alcanza de forma segura únicamente cuando **la sumatoria del procesamiento local y el reportado por las otras réplicas iguala al total esperado**.

A nivel interno, la concurrencia entre el hilo principal y el de control se gestiona mediante **Locks**, que protegen los diccionarios en memoria y evitan inconsistencias. Asimismo, ante una señal del sistema (**SIGTERM**), se ejecuta un **Graceful Shutdown** que detiene el consumo de colas, finaliza los hilos con *join* y libera los recursos de red de forma ordenada. En particular, la detención del consumo del *Control Exchange* se realiza de forma *thread-safe* utilizando `add_callback_threadsafe`, garantizando que `stop_consuming` se ejecute correctamente en el *thread* correspondiente al *consumer*.

Por otro lado, la etapa de **Aggregation** escala mediante **sharding determinístico** aplicado sobre el nombre de la fruta. Esta estrategia no solo distribuye uniformemente la carga entre réplicas (al no agrupar por cliente), sino que también simplifica la lógica del nodo **Join**, ya que garantiza que todas las ocurrencias de una misma entidad sean procesadas por el mismo nodo, asegurando consistencia en los conteos.

Para coordinar la finalización y la emisión de resultados, tanto los nodos de **Aggregation** como el nodo **Join** implementan **barreras lógicas** basadas en la recepción de finalizaciones. En **Aggregation**, la barrera espera los *EOF* de todos los nodos **Sum** antes de emitir resultados parciales. Finalmente, el nodo **Join** replica este mecanismo, asegurando que el cálculo del top por cliente se realice únicamente cuando todas las etapas previas del pipeline han finalizado.